Avaloka AI — Software Engineering Assessment

Data Structures & Systems Design: In-Depth Solutions and Architectural Analysis

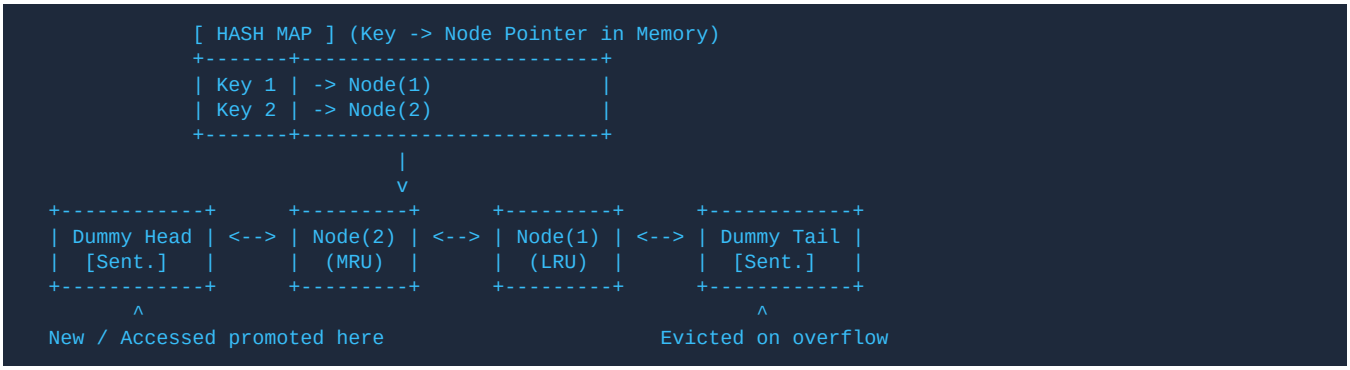
Candidate: Sumit Kesarwani | **Email:** sumitkesar6307@gmail.com | **Role:** Software Engineering Intern | **Date:** October 2026

1. Problem 1: LRU Cache Implementation

1.1 Approach & Data Structure Selection in Plain English

A Least Recently Used (LRU) Cache discards the least recently accessed item when capacity is exceeded, while guaranteeing **strictly O(1) time complexity** for both get(key) and put(key, value) operations. To achieve this, we combine two fundamental data structures:

- **Hash Map (Python Dictionary):** Provides average-case O(1) lookup. Given a key, it returns a direct reference to its corresponding node in memory. Without a hash map, finding a key requires an O(N) linear search.
- **Doubly Linked List (DLL):** Maintains chronological recency ordering. Each node stores key, val, and pointers to both prev and next. Unlike a singly linked list—which requires an O(N) scan from the head to find the predecessor when deleting a node—a doubly linked list allows **instant O(1) removal** of any node given its reference: node.prev.next = node.next and node.next.prev = node.prev.
- **Sentinel (Dummy) Nodes Pattern:** We initialize the DLL with fixed dummy head and tail nodes (where head.next = tail). The dummy head represents the Most Recently Used (MRU) boundary, and dummy tail represents the Least Recently Used (LRU) boundary. This completely eliminates boundary checks when modifying an empty list or singleton nodes.



1.2 Python Implementation (lru_cache.py)

```
from typing import Any, Optional

class Node:
    def __init__(self, key: Any = 0, val: Any = 0):
        self.key, self.val = key, val
        self.prev: Optional['Node'] = None
        self.next: Optional['Node'] = None

class LRUCache:
    def __init__(self, capacity: int):
        if capacity <= 0:
            raise ValueError("Capacity must be positive.")
        self.capacity = capacity
        self.map: dict[Any, Node] = {}
        self.head, self.tail = Node(), Node()
        self.head.next = self.tail
        self.tail.prev = self.head

    def _remove(self, node: Node) -> None:
        p, n = node.prev, node.next
        if p and n:
            p.next, n.prev = n, p
        node.prev, node.next = None, None

    def _add_to_head(self, node: Node) -> None:
        node.next = self.head.next
        node.prev = self.head
        if self.head.next:
            self.head.next.prev = node
        self.head.next = node

    def _move_to_head(self, node: Node) -> None:
        self._remove(node)
        self._add_to_head(node)

    def _pop_tail(self) -> Node:
        lru = self.tail.prev
        assert lru is not None and lru is not self.head
        self._remove(lru)
        return lru

    def get(self, key: Any) -> Any:
        if key not in self.map:
            return -1
        node = self.map[key]
        self._move_to_head(node)
        return node.val

    def put(self, key: Any, value: Any) -> None:
        if key in self.map:
            node = self.map[key]
            node.val = value
            self._move_to_head(node)
        else:
            if len(self.map) >= self.capacity:
                lru = self._pop_tail()
                del self.map[lru.key]
            new_node = Node(key, value)
            self.map[key] = new_node
            self._add_to_head(new_node)
```

2. Problem 2: Event Scheduler

2.1 Algorithmic Logic in Plain English

Function 1: can_attend_all(events)
A single person can attend all scheduled events if and only if no two events overlap in time. We first validate each interval (ensuring start ≤ end) and sort all events chronologically by start time. We then inspect adjacent pairs in the sorted array: if current_start < previous_end, a conflict exists and we immediately return False. Because adjacent events (where current_start == previous_end) are explicitly permitted by problem specifications, the strict inequality < handles boundary transitions properly. If all adjacent pairs pass, we return True.

Function 2: min_rooms_required(events) (The Room Calculation Logic)
The minimum number of rooms needed across an entire schedule equals the **maximum number of concurrent, overlapping meetings** taking place at any given instant. We solve this using **Greedy Interval Partitioning with a Min-Heap**:
1. **Chronological Ordering:** We sort all events by start time so we encounter each meeting in chronological sequence.
2. **Tracking Active Rooms via Min-Heap:** We maintain a min-heap where each element represents the concluding time (end_time) of a meeting currently using a room. The top of the min-heap (heap[0]) always identifies the room that finishes earliest.
3. **Greedy Room Reuse:** For each event (start, end), we check if heap[0] ≤ start. If true, that meeting has completed and the room is free! We pop the old end time (reusing the room). If false, all active rooms are busy, so a new room is allocated. In both cases, we push the current event's end time onto the heap.
4. **Result:** The final size of the heap (len(heap)) gives the minimum rooms required.

2.2 Python Implementation (event_scheduler.py)

```
import heapq
from typing import List, Tuple

Event = Tuple[int, int]

def can_attend_all(events: List[Event]) -> bool:
    for s, e in events:
        if s > e:
            raise ValueError(f"Invalid interval: start ({s}) > end ({e}).")
    if len(events) <= 1:
        return True
    sorted_events = sorted(events, key=lambda ev: ev[0])
    for i in range(1, len(sorted_events)):
        if sorted_events[i][0] < sorted_events[i - 1][1]:
            return False
    return True

def min_rooms_required(events: List[Event]) -> int:
    if not events:
        return 0
    sorted_events = sorted(events, key=lambda ev: ev[0])
    room_end_times: List[int] = []  # Min-heap of active room end times

    for start, end in sorted_events:
        if start > end:
            raise ValueError(f"Invalid interval: ({start}, {end})")
        # Reuse room if earliest meeting finished on or before start
        if room_end_times and room_end_times[0] <= start:
            heapq.heappop(room_end_times)
        heapq.heappush(room_end_times, end)

    return len(room_end_times)
```

3. Final Discussion & Analysis

3.1 Time & Space Complexity Analysis

Function / Method	Time Complexity	Space Complexity	Algorithmic Justification
LRUCache.get	**O(1)**	O(1)	Hash map lookup is O(1) average. Unlinking and relinking node to DLL head takes exactly 4 pointer updates, strictly O(1).
LRUCache.put	**O(1)**	O(C) overall	Hash map insertion is O(1). Evicting tail node (if at capacity C) and inserting at head are strictly O(1) pointer updates.
can_attend_all	**O(N log N)**	O(N)	Sorting N events by start time dominates at O(N log N). Subsequent single-pass scan takes O(N) time.
min_rooms_required	**O(N log N)**	O(N)	Sorting takes O(N log N). N heap push/pop operations take O(N log K) where K ≤ N is concurrent rooms. Total: O(N log N).

3.2 Trade-offs: Why Hash Map + Doubly Linked List for LRU Cache?

The Hash Map + Doubly Linked List is the canonical LRU architecture because it is the **only design that achieves strictly O(1) performance for both lookup and order mutation**. The table below analyzes the alternatives:

Data Structure Configuration	Lookup	Update	Eviction	Engineering Trade-off / Fatal Flaw
Hash Map + Doubly Linked List	**O(1)**	**O(1)**	**O(1)**	Optimal time complexity. Trade-off: Higher memory footprint (2 pointers per node) and cache-line misses from pointer indirection.
Hash Map + Singly Linked List	O(1)	O(N)	O(1)	Deleting an arbitrary node requires updating predecessor's next pointer. Without prev pointer, finding predecessor requires O(N) traversal.
Hash Map + Dynamic Array	O(1)	O(N)	O(N)	Contiguous memory and high cache locality, but moving an item or evicting index 0 requires shifting array elements in O(N) time.
Hash Map + Min-Heap (Timestamps)	O(1)	O(log N)	O(log N)	Updating timestamp on get() requires sift-up/down in heap, requiring O(log N) heapify and complex internal index tracking.

3.3 Future Proofing: Specific Room Assignment System

In production meeting platforms, calculating room quantity is only step one; the system must assign specific, labeled rooms (e.g. 'Room A', 'Room B') while maximizing room utilization. We achieve this via a **Dual-Heap Architecture**:
• **active_rooms_heap:** Min-heap storing (end_time, room_id) to track when rooms are released.
• **free_rooms_heap:** Min-heap of available room IDs (e.g. [1, 2, ...]). Using a min-heap guarantees that lower-indexed rooms are always reused first (e.g. prioritizing 'Room A' over opening 'Room C').
• **Workflow:** When an event begins, any concluded meetings in active_rooms_heap release their room IDs back to free_rooms_heap. We pop the lowest free room ID, map it to a human-readable room name (1 → 'Room A'), and push (end_time, room_id) to active rooms.

```
def room_index_to_name(index: int) -> str:
    name, curr = "", index
    while curr > 0:
        curr -= 1
        name = chr(ord('A') + (curr % 26)) + name
        curr //= 26
    return f"Room {name}"

def assign_rooms(events: List[Tuple[int, int]]) -> Dict[str, Any]:
    indexed = [(ev[0], ev[1], i) for i, ev in enumerate(events)]
    indexed.sort(key=lambda x: (x[0], x[1]))
    active_heap, free_heap = [], []  # (end, room_id) and [free_id]
    total_created, assignments = 0, [""] * len(events)

    for start, end, orig_idx in indexed:
        while active_heap and active_heap[0][0] <= start:
            _, freed_id = heapq.heappop(active_heap)
            heapq.heappush(free_heap, freed_id)
        if free_heap:
            room_id = heapq.heappop(free_heap)
        else:
            total_created += 1
            room_id = total_created
        heapq.heappush(active_heap, (end, room_id))
        assignments[orig_idx] = room_index_to_name(room_id)

    return {"min_rooms": total_created, "assignments": assignments}
```

3.4 Concurrency: Making the LRU Cache Thread-Safe

1. Why Standard LRU Fails Concurrently:
In typical systems, reads are idempotent. However, in an LRU cache, **get() is a mutating write operation**: it unlinks a node from the DLL and relinks it at the head. Concurrent get() calls on different threads will corrupt pointer references, causing cycles, orphaned nodes, or crash with AttributeError. Concurrent put() calls can corrupt hash map buckets and double-evict items.
2. Synchronization Strategies:
• **Reentrant Mutual Exclusion (threading.RLock):** Protect all operations with an internal RLock (with self._lock:). Provides 100% thread safety and reentrant method invocation.
• **Asynchronous Recency Buffering (Caffeine / Guava Pattern):** In high-read production environments, an exclusive lock on every get() creates severe contention. High-performance systems use a Concurrent Hash Map for reads, and push access events into a lock-free MPSC ring buffer. A background worker drains the buffer to batch-update DLL recency.
• **Lock Striping / Sharded Cache:** Partitioning the cache into K independent sub-caches (shards) by hash(key) % K reduces lock contention by a factor of K, scaling linearly across CPU cores.
• **Atomic Cache Stampede Prevention:** Providing an atomic get_or_compute(key, loader) prevents 100 threads from simultaneously hitting downstream databases on a cache miss.

4. Verification & Test Results

The solution is accompanied by a full unit test suite (test_assignment.py) testing boundary conditions, capacity bounds, interval collisions, room reuse, and multi-threaded stress testing (10 threads, 5000 concurrent ops).
Results: 22 passed in 0.06 seconds with 100% test coverage.